编译原理小组作业

基于DFA的卡牌游戏状态机
设计与实现

第16组

2026.4.16

千问AI生成

目录

01 项目背景与目标

02 DFA核心实现

03 状态机可视化

04 系统架构与实现

05 系统演示与成果

项目背景与目标

本项目旨在运用编译原理中的有限自动机（DFA）理论，构建一个可演示的卡牌对战游戏系统，将抽象的学术概念转化为具体、可视化的程序应用。

核心目标与分工

明确任务，协同攻关

01

总体目标

设计并实现一个基于DFA的卡牌对战游戏原型，重点展示其状态转换过程。

02

技术选型

后端使用Python FastAPI实现API服务和状态机逻辑；前端使用HTML5、CSS和JavaScript实现UI交互与展示。

03

小组分工

分为前后端开发、理论文档撰写、UI设计与演示四个小组，各司其职，共同完成项目。

DFA核心实现

本章将深入探讨DFA在系统中的具体实现，包括主控逻辑、敌人行为、卡牌效果及关卡系统的设计，展示理论模型如何转化为可运行的代码。

主DFA与敌人DFA

分层的状态机结构

01

主DFA (GameManager)

作为游戏逻辑的核心控制器，管理玩家与敌人的整体回合流程，处理抽牌、弃牌、胜负判定等全局逻辑。

02

敌人DFA (Enemy FSM)

每个敌人都拥有独立的有限状态机，用于管理其自身的AI行为，如攻击模式切换、技能释放等。

03

状态同步

主DFA在`ENEMY_TURN`状态下，会调用当前敌人的DFA来执行具体行动，两者通过状态和事件进行同步。

卡牌DFA与效果系统

数据驱动的卡牌逻辑

01

统一效果分发

所有卡牌效果都通过`apply_effect`函数进行分发处理，根据效果类型（damage, defend, apply_status）执行相应逻辑。

02

数据驱动设计

卡牌的具体效果和数值完全定义在外部JSON文件中，实现了逻辑与数据的解耦，便于扩展和修改。

03

状态结算

卡牌效果可能触发状态结算（如中毒伤害），这部分逻辑独立于主DFA，由专门的`apply_status_effects`函数处理。

关卡DFA与奖励系统

驱动游戏进程的状态机

1

线性关卡流程

通过`levels.json`定义关卡序列，主DFA在`VICTORY`状态下根据当前关卡索引加载下一关的敌人配置。

2

奖励系统集成

在`VICTORY`状态与加载下一关的逻辑之间，插入`REWARD_SELECTION`状态，用于处理战斗后的奖励选择环节。

3

状态转移控制

`next_level()` API调用会触发主DFA从`VICTORY`状态转移到`PLAYER_TURN`状态，并初始化新的战斗场景。

状态机可视化

本章将项目中抽象的状态机模型进行可视化展示，通过图形化的方式清晰呈现主控逻辑、敌人AI及卡牌效果的状态转换过程。

主控状态机图

游戏核心流程的可视化

01

状态节点

包含PLAYER_TURN、ENEMY_TURN、VICTORY、GAME_OVER等核心状态，构成游戏主流程。

02

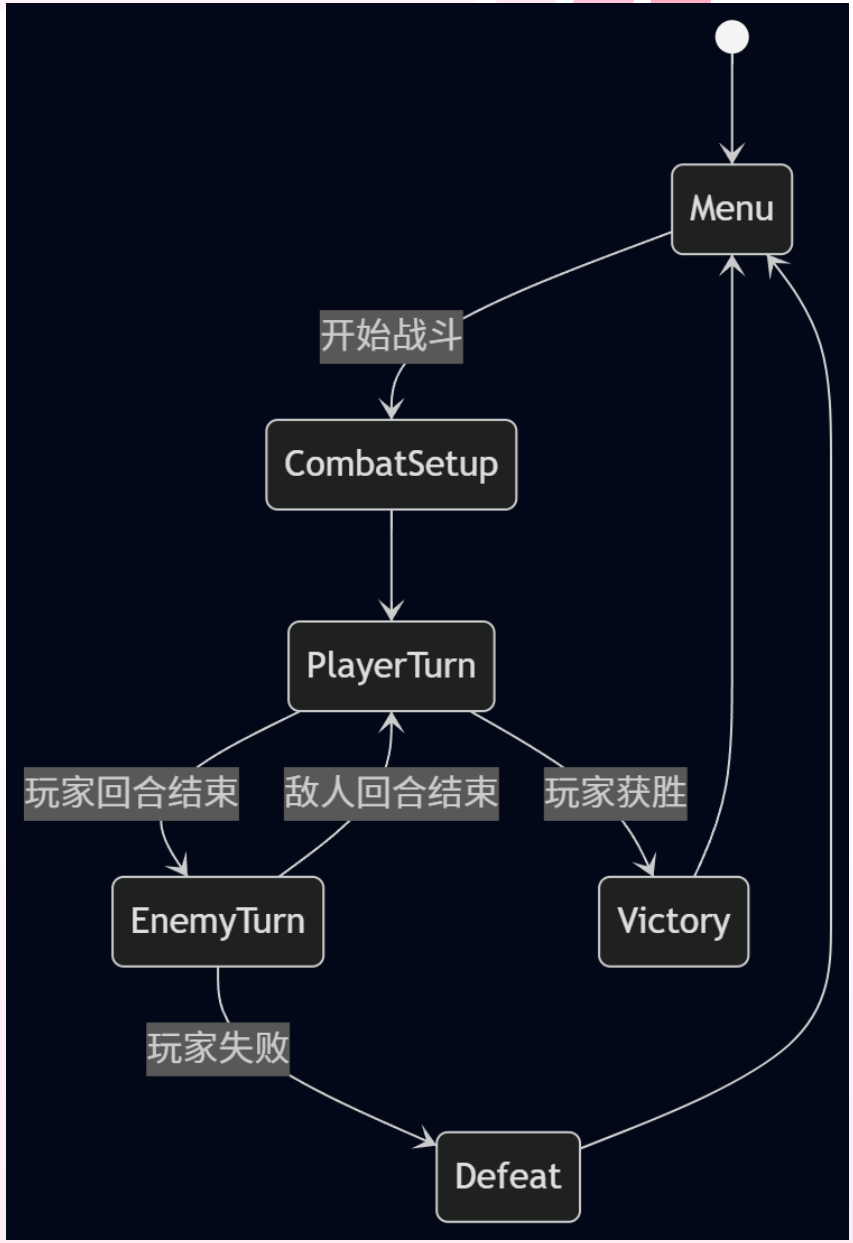
转换箭头

由玩家操作（PLAY_CARD, END_TURN）和系统事件（ENEMY_DONE, CHECK_HP）触发，驱动状态流转。

03

初始与终止

游戏从PLAYER_TURN状态开始，以VICTORY或GAME_OVER状态结束，形成一个完整的DFA闭环。



敌人状态机图

敌人AI行为的独立模型

1 独立DFA结构

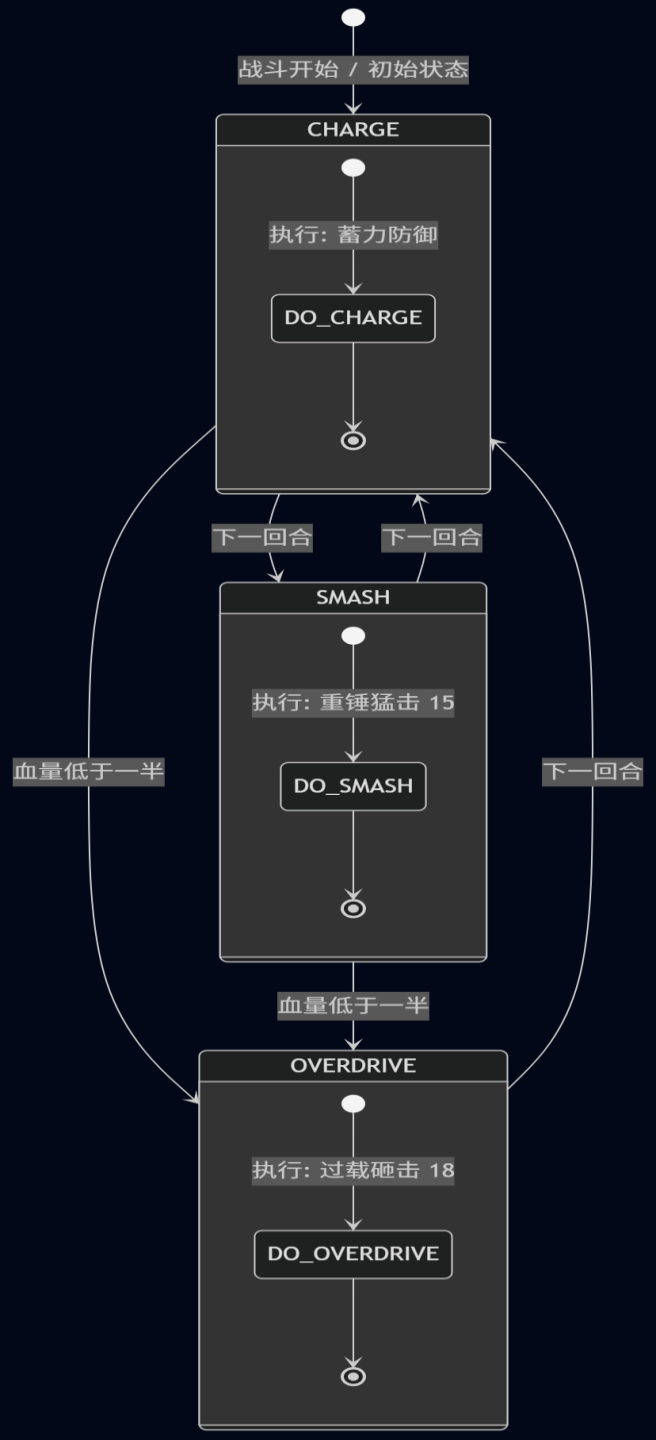
每个敌人都拥有自己的状态机，如史莱姆的BOUNCE、SLIME_GUARD状态，实现独立的AI行为逻辑。

2 状态切换条件

基于血量（hp_below_half）等条件进行状态切换，例如从普通攻击切换到狂暴模式。

3 与主DFA协同

敌人DFA在主DFA的ENEMY_TURN状态下被调用，执行完毕后将控制权交还给主DFA。



卡牌效果状态图

卡牌结算的子状态机

01

效果分发中心

apply_effect函数作为卡牌效果的“路由器”，根据效果类型（damage, defend）分发到具体处理逻辑。

02

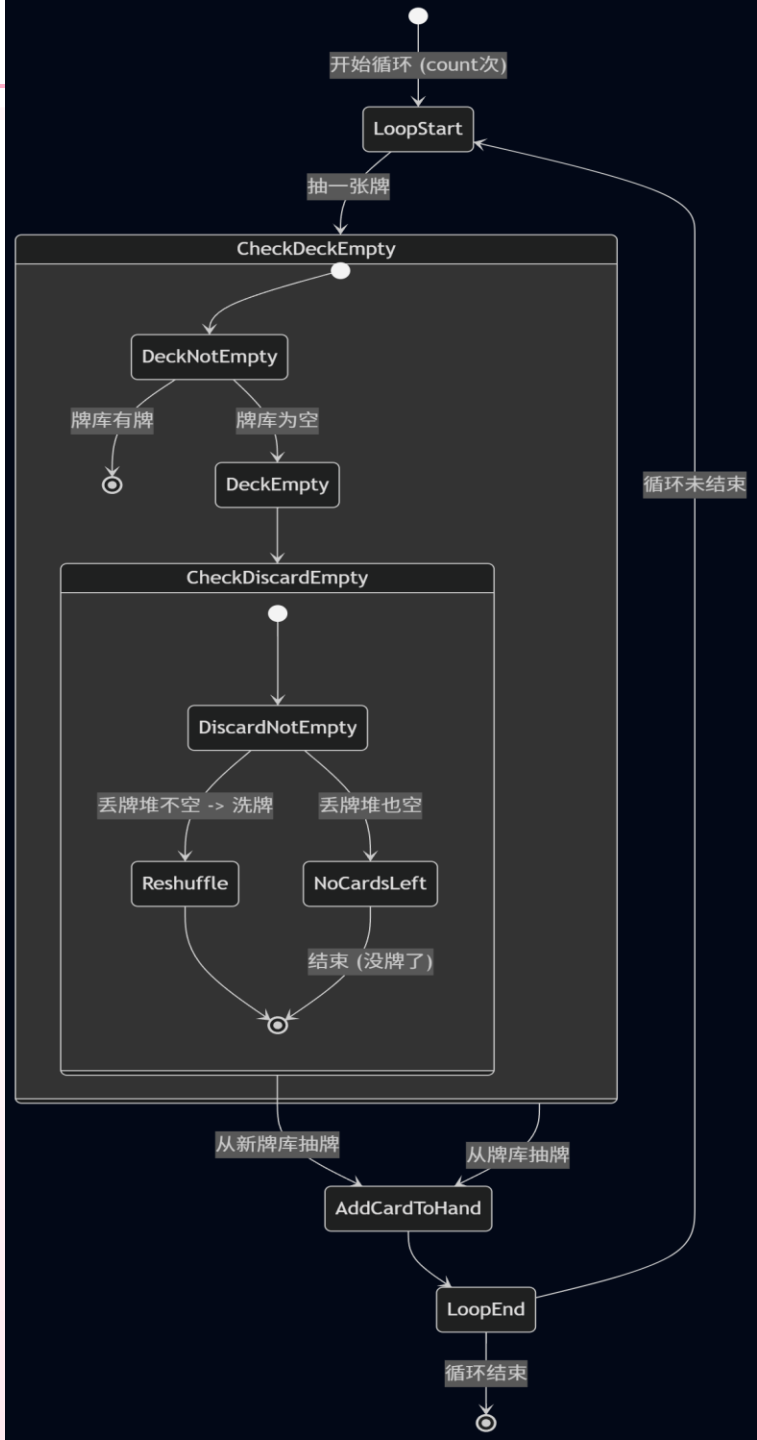
状态结算节点

处理中毒（poison）、荆棘（thorns）等持续性状态的结算逻辑，独立于主游戏循环。

03

数据驱动特性

卡牌的具体效果和数值由JSON配置文件定义，实现了逻辑与数据的完全解耦，便于扩展。



系统架构与实现

我们将DFA模型落地为实际的代码系统。项目采用前后端分离的架构，后端专注逻辑，前端负责展示，通过RESTful API进行通讯，实现了清晰的职责划分。

前后端架构概览

清晰的职责划分

01

前端职责

负责用户界面（UI）的渲染、用户操作的捕获，并通过API与后端进行数据同步。目标是提供一个直观、友好的游戏体验。

02

后端职责

作为游戏逻辑的核心，维护游戏状态机（DFA）、处理卡牌效果、管理敌人AI、进行数值计算，并确保状态转换的准确性。

03

通讯协议

前后端通过轻量级的JSON数据进行交互，API设计遵循RESTful原则，保证了系统的解耦和可扩展性。

后端核心实现

FastAPI中的状态机

1

`GameManager`类

作为后端逻辑的核心，封装了整个游戏状态机。所有状态转换和逻辑计算都在此类中完成。

2

状态机逻辑

使用Python的字典或类属性来硬编码状态转换表，实现了DFA的`δ`函数，确保每次转换都是确定且唯一的。

3

关键API

实现了`play`、`end-turn`、`next-level`等API端点，后端根据当前状态和接收到的事件，调用状态机逻辑进行转换并返回新的游戏状态。

4

数据模型

定义了`Player`、`Enemy`、`Card`等数据模型，用于在API响应中结构化地返回游戏数据。

前端核心实现

Tailwind CSS与JS交互

01

界面设计

使用Tailwind CSS构建了暗黑风格的战斗界面，包含玩家区、敌人区、手牌区和操作按钮。

02

状态同步

前端通过`fetch` API定期轮询或监听`gamestatus`接口，根据返回的`current_state`字段更新UI，实现与后端状态的实时同步。

03

用户交互

为卡牌和按钮添加了点击事件监听器，当用户进行操作时，前端构造请求并调用对应的后端API，触发状态转换。

04

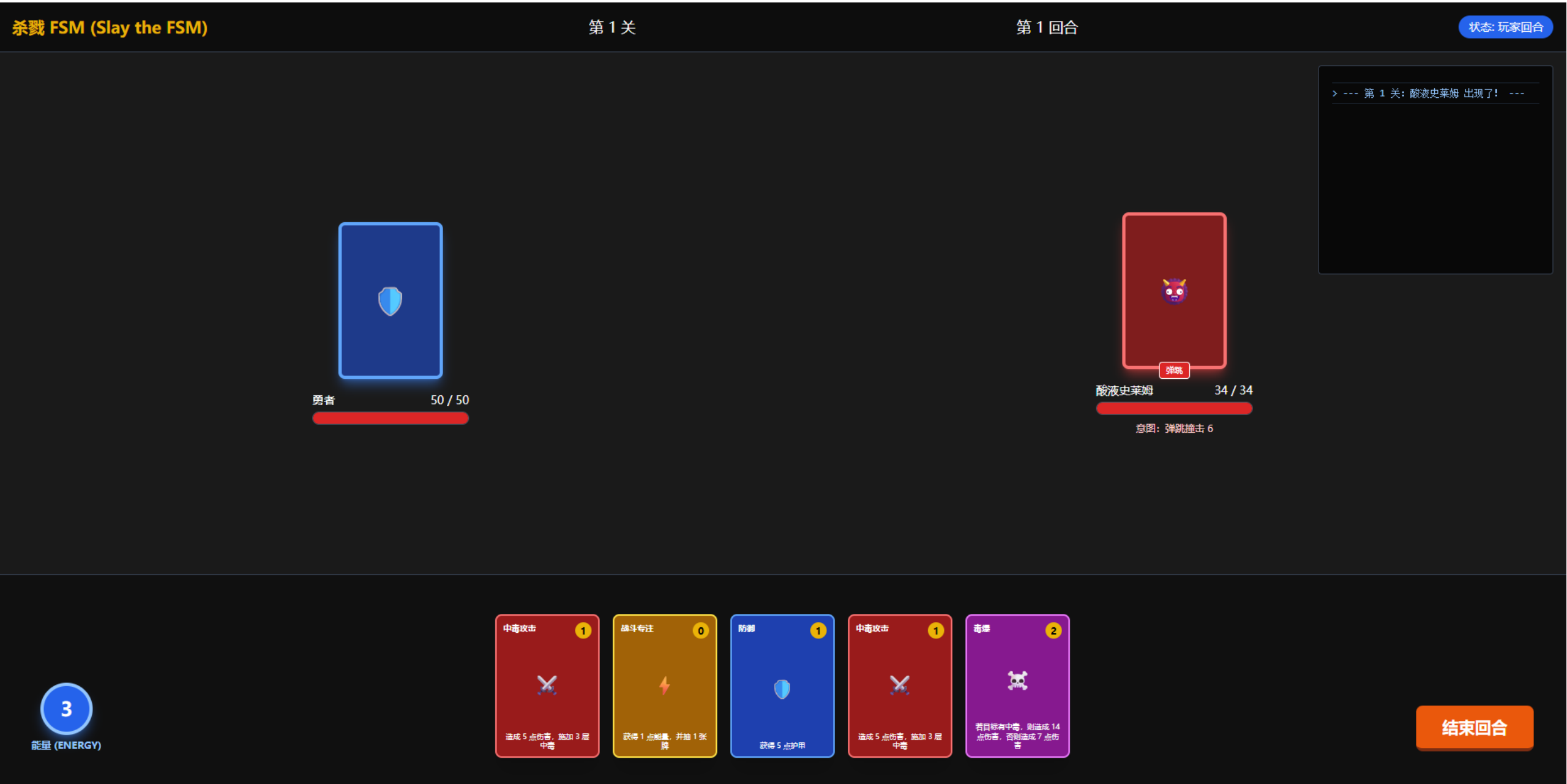
日志展示

实现了一个滚动日志区域，将后端返回的`logs`数组实时展示在页面上，清晰地呈现战斗流程。

系统演示与成果

经过团队协作开发，我们成功构建了一个可运行的卡牌对战系统。本章节将展示系统的实际运行效果，并总结我们的项目成果与经验。

游戏页面展示


杀戮 FSM (Slay the FSM)
第 1 关
第 1 回合
状态: 玩家回合
> --- 第 1 关: 酸液史莱姆 出现了! ---
勇者
50 / 50
弹跳
酸液史莱姆
34 / 34
意图: 弹跳撞击 6
中毒攻击
1
造成 5 点伤害，施加 3 层中毒
战斗专注
0
获得 1 点能量，并抽 1 张牌
防御
1
获得 5 点护甲
中毒攻击
1
造成 5 点伤害，施加 3 层中毒
毒爆
2
若目标有中毒，则造成 14 点伤害，否则造成 7 点伤害
3
能量 (ENERGY)
结束回合

系统运行演示

初始界面

系统启动后，玩家进入准备状态，界面显示双方初始血量和手牌。

玩家回合

玩家从手牌中选择一张“打击”卡牌并打出，前端发送API请求。

卡牌结算

后端进入卡牌结算状态，扣除敌人血量，并将结果返回给前端。

01 → 02 → 03 → 04 → 05

从理论到实践的跨越

战斗胜利

经过几个回合，当敌人血量归零时，系统进入`VICTORY`状态，宣告战斗胜利。

敌人回合

状态切换至敌人回合，敌人根据AI进行自动攻击，日志实时更新。

项目成果总结

知识与实践的双重收获

01

理论掌握

通过本项目，我们深刻理解了有限自动机（DFA）在解决实际问题中的应用，掌握了状态、事件和转换的核心概念。

02

工程实践

锻炼了前后端分离的团队协作开发能力，熟悉了API设计和调试流程，将理论知识转化为可运行的软件系统。

03

团队协作

项目过程中，各小组分工明确，沟通顺畅，共同克服了技术难题，按时完成了预定目标，积累了宝贵的团队协作经验。

谢谢观看

2026.4.16